

Multithreaded programming

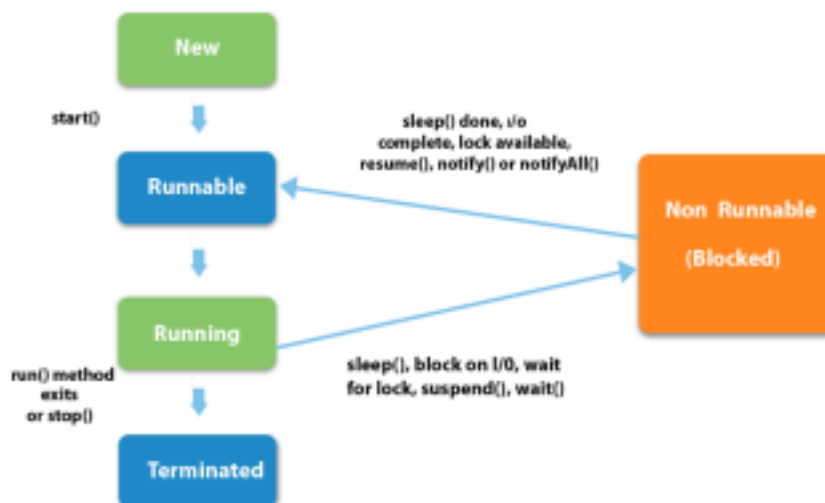
Java provides built-in support for multithreaded programming. A multithreaded program contains two or more parts that can run concurrently. Each part of such a program is called a thread. In a thread-based multitasking environment, single program can perform two or more tasks simultaneously. For instance, a text editor can format text at the same time that it is printing, as long as these two actions are being performed by two separate threads.

Multitasking threads require less overhead than multitasking processes. Processes are heavyweight tasks that require their own separate address spaces. Interprocess communication is expensive and limited. Context switching from one process to another is also costly. Threads, on the other hand, are lighter weight. They share the same address space and cooperatively share the same heavyweight process. Interthread communication is inexpensive, and context switching from one thread to the next is lower in cost. Multithreading enables the user to write efficient programs that make maximum use of the processing power available in the system.

~~Life cycle of a thread~~ (imp)

A thread can be in one of the five states.

1. New
2. Runnable
3. Running
4. Non-Runnable (Blocked)
5. Terminated



The thread will be in **new** state if an instance of thread is created but the `start()` method is

yet to be invoked. After the invocation of the `start()` method the thread moves to the **runnable** state. At this state the thread is ready to run but is not selected by the thread scheduler. The thread moves to the **running** state after it is selected by the thread scheduler to be run. The thread moves to the **blocked** state when the thread is still alive but not eligible to run due to reasons like block on I/O or sleep. When the thread exits from the `run()` method it is said to be in the **terminated** state.

Thread Priorities

Java's multithreading system is built upon the `Thread` class, its methods, and its companion interface, `Runnable`. The `Thread` class defines several methods that help manage threads. **`getName()`** - Obtain a thread's name.

`getPriority()` - Obtain a thread's priority.

`isAlive()` - Determine if a thread is still running. ✓

`join()` - Wait for a thread to terminate. ✓

`run()` - Entry point for the thread.

`sleep()` - Suspend a thread for a period of time. ✓

`start()` - Start a thread by calling its `run` method. ✓

The Main Thread

When a Java program starts up, one thread begins running immediately. That is the *main thread* of the program. It is the thread from which other "child" threads will be spawned. It must be the last thread to finish execution because it performs various shutdown actions. Although the main thread is created automatically when the program is started, it can be controlled through a **`Thread`** object. A reference to it must be obtained by calling the method **`currentThread()`**, which is a **public static** member of **`Thread`**.

```
public class CurrentThreadDemo {  
  
    public static void main(String args[]) {  
  
        Thread t = Thread.currentThread();  
        System.out.println("Current thread: " + t);  
  
        t.setName("My Thread");  
        System.out.println("After name change: " + t);  
  
        try {  
            for(int n = 5; n > 0; n--) {
```

```

        System.out.println(n);
        Thread.sleep(1000);
    }

    catch (InterruptedException e) {
        System.out.println("Main thread interrupted");
    }
}
}

```

In this program, a reference to the current thread is obtained by calling **currentThread()**, and this reference is stored in the local variable **t**. The program displays information about the thread. The program then calls **setName()** to change the internal name of the thread. Information about the thread is then redisplayed. Next, a loop counts down from five, pausing one second between each line. The pause is accomplished by the **sleep()** method. The argument to **sleep()** specifies the delay period in milliseconds. The **sleep()** method in **Thread** might throw an **InterruptedException** and hence has to be handled appropriately.

Output:

```

Current thread: Thread[main,5,main]
After name change: Thread[My Thread,5,main]
5
4
3
2
1

```

A Thread object when passed to the print method displays : the name of the thread, its priority, and the name of its group.

Creating a Thread

A thread can be created by :

- Implementing the Runnable interface
- Extending the Thread class

Runnable interface 

Step 1 : Create a class that implements Runnable interface

Step 2 : Override the public void run method to include the required logic for a thread.

(In the main())

Step 3 : Create an object of the class that implemented Runnable.

Step 4 : Create an object of Thread class and pass the object created in Step 3 in the Thread constructor.

Step 5: Call the start() method using the Thread object.

```
public class NewThread_Runnable implements Runnable{

    public void run() {

        try {

            for(int i = 5; i > 0; i--) {

                System.out.println("Child Thread: " + i);
                Thread.sleep(500);

            }

        }

        catch (InterruptedException e) {

            System.out.println("Child interrupted.");

        }

        System.out.println("Exiting child thread.");

    }

    public static void main(String args[]){

        NewThread_Runnable rn = new NewThread_Runnable();
        Thread t1 = new Thread(rn);
        t1.start();

        try {

            for(int i = 5; i > 0; i--) {

                System.out.println("Main Thread: " + i);
                Thread.sleep(1000);

            }

        }

        catch (InterruptedException e) {

            System.out.println("Main thread interrupted.");

        }

        System.out.println("Main thread exiting.");

    }

}
```

Extending Thread *(only keyword)*

Step 1 : Create a class that extends Thread class.

Step 2 : Override the public void run method to include the required logic for a thread.

(In the main())

Step 3 : Create an object of the class that extended Thread.

Step 4 : Call the start() method using the object created in Step 3. public class

```
NewThread_Thread extends Thread {
```

```

public void run()
{
    try {
        for(int i = 5; i > 0; i--)
        {
            System.out.println("Child Thread: " + i);
            Thread.sleep(500);
        }
    }

    catch (InterruptedException e) {
        System.out.println("Child interrupted.");
    }
    System.out.println("Exiting child thread.");
}

public static void main(String args[])
{
    NewThread_Thread t1 = new NewThread_Thread();
    t1.start();

    try {
        for(int i = 5; i > 0; i--)
        {
            System.out.println("Main Thread: " + i);
            Thread.sleep(1000);
        }
    }

    catch (InterruptedException e) {
        System.out.println("Main thread interrupted.");
    }

    System.out.println("Main thread exiting.");
}
}

```

Creating Multiple Threads:

The above examples use only two threads: the main thread and one child thread. However, a program can spawn as many threads as needed. An example is shown below.

```

public class One extends Thread {

    public void run()
    {
        try
        {
            int i=0;
            while (i<5)
            {
                Thread.sleep(500);
                System.out.println("Good morning");
            }
        }
    }
}

```

```

        i++;
    }
}
catch(InterruptedException e)
{
    System.out.println("Interrupted");
}
}
}

```

```

public void run()
{
    try
    {
        int i=0;
        while (i<5)
        {
            Thread.sleep(500);
            System.out.println("Hello");
            i++;
        }
    }

    catch(InterruptedException e)
    {
        System.out.println("Interrupted");
    }
}

public class Three extends Thread {

    public void run()
    {
        try
        {
            int i=0;
            while (i<5)
            {
                Thread.sleep(500);
                System.out.println("Welcome");
                i++;
            }
        }

        catch(InterruptedException e)
        {
            System.out.println("Interrupted");
        }
    }
}
}
}

```

```

public class MyThread {

    public static void main(String[] args) {

```

```

        One t1=new One();
        Two t2=new Two();
        Three t3=new Three();
        t1.start();
        t2.start();
        t3.start();

    try{
        Thread.sleep(5000);

    }

    catch(InterruptedException e)
    {
        System.out.println("Interrupted");
    }

}

```

Using `isAlive()` and `join()`

In order to determine if a thread has finished execution, the **`isAlive()`** method of the `Thread` class is used. The **`isAlive()`** method returns **`true`** if the thread upon which it is called is still running. It returns **`false`** otherwise. The `join()` method waits until the thread on which it is called terminates. `join()` throws an `InterruptedException` and has to be appropriately handled.

```

public class IsAliveJoin extends Thread {

    public void run()
    {
        try {

            for(int i = 3; i > 0; i--)
            {

                System.out.println("Child Thread: " + i);
                Thread.sleep(500);
            }

        }

        catch (InterruptedException e) {
            System.out.println("Child interrupted.");
        }
    }
}

```

```

    }

    System.out.println("Exiting child thread."+Thread.currentThread().getName());
}

public static void main(String args[])
{

    IsAliveJoin t1 = new IsAliveJoin();
    t1.start();

    IsAliveJoin t2 = new IsAliveJoin();
    t2.start();

    IsAliveJoin t3 = new IsAliveJoin();
    t3.start();

    System.out.println("Thread 1 is alive "+t1.isAlive());
    System.out.println("Thread 2 is alive "+t2.isAlive());
    System.out.println("Thread 3 is alive "+t3.isAlive());

    try {

        System.out.println("Waiting for threads to finish.");

        t1.join();
        t2.join();
        t3.join();

    }
    catch (InterruptedException e) {
        System.out.println("Main thread Interrupted");
    }

    System.out.println("Thread 1 is alive "+t1.isAlive());
    System.out.println("Thread 2 is alive "+t2.isAlive());
    System.out.println("Thread 3 is alive "+t3.isAlive());

    System.out.println("Main thread exiting.");

}

}

```


Thread Priorities

Thread priorities are used by the thread scheduler to decide when each thread should be allowed to run. In theory, higher-priority threads get more CPU time than lower-priority threads. In practice, the amount of CPU time that a thread gets often depends on several factors besides its priority. Each thread has a priority. Priorities are represented by a number between 1 and 10.

Methods that handle Thread priorities:

public final int getPriority(): The `java.lang.Thread.getPriority()` method returns the priority of the given thread.

public final void setPriority(int newPriority): The `java.lang.Thread.setPriority()` method updates or assigns the priority of the thread to `newPriority`. The method throws `IllegalArgumentException` if the value `newPriority` goes out of the range, which is 1 (minimum) to 10 (maximum).

There are 3 constants defined in Thread class:

1. `public static int MIN_PRIORITY`
2. `public static int NORM_PRIORITY`
3. `public static int MAX_PRIORITY`

Default priority of a thread is 5 (`NORM_PRIORITY`). The value of `MIN_PRIORITY` is 1 and the value of `MAX_PRIORITY` is 10.

Synchronization *(imp)*

When two or more threads need access to a shared resource, they need some way to ensure that the resource will be used by only one thread at a time. The process by which this is achieved is called **synchronization**. Java provides unique, language-level support for it.

The key to synchronization is the concept of the monitor. A monitor is an object that is used as a lock. Only one thread can own a monitor at a given time. When a thread acquires a lock, it is said to have entered the monitor. All other threads attempting to enter the locked monitor will be suspended until the first thread exits the monitor.

What happens when there is no synchronization? In the example below, there is no synchronization, so output is inconsistent.

```
public class Table {  
    void printTable(int n){  
        for(int i=1;i<=5;i++){  
            System.out.println(n*i);  
            try{  
                Thread.sleep(400);  
            } catch (InterruptedException e){  
                System.out.println(e);}  
        }  
    }  
}
```

```
}
```

```
class MyThread1 extends Thread{  
    Table t;  
    MyThread1(Table t){  
        this.t=t;  
    }  
    public void run(){  
        t.printTable(5);  
    }  
}
```

```
}
```

```
class MyThread2 extends Thread{  
    Table t;  
    MyThread2(Table t){  
        this.t=t;  
    }  
    public void run(){  
        t.printTable(100);  
    }  
}
```

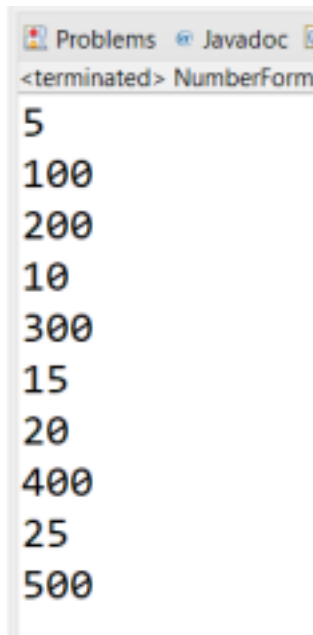
```
class TestSynchronization1 {  
    public static void main(String args[]){  
        Table obj = new Table();//only one object
```

```

MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj);
t1.start();
t2.start();
    }
}

```

Output:



Since there no synchronization, the output is inconsistent. That is, the multiples of 2 and 100 are printed out randomly. If there is a need to print the multiples of 2 entirely first then followed by the multiples of 10, then the printTable method has to be written as a synchronized method. In such a case the thread when it is running the synchronized method acquires a lock for the same and does not allow other threads to access the method till its run is complete.

```

public class Table {
    synchronized void printTable(int n){
        for(int i=1;i<=5;i++){
            System.out.println(n*i);
            try{
                Thread.sleep(400);
            } catch (InterruptedException e){
                System.out.println(e);}
        }
    }
}

```

```

class MyThread1 extends Thread{ Table t;
    MyThread1(Table t){ this.t=t;
    }
    public void run(){
        t.printTable(5);
    }
}

```

```

class MyThread2 extends Thread{ Table t;
    MyThread2(Table t){ this.t=t;
    }
    public void run(){
        t.printTable(100);
    }
}

```

```

class TestSynchronization1 { public static void
main(String args[]){ Table obj = new Table();//only
one object MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj); t1.start();
t2.start();
    }
}

```

Output:

```
Problems Javadoc
<terminated> NumberForm
5
10
15
20
25
100
200
300
400
500
```

As seen above, the output is synchronized because of the use of the synchronized keyword on the method printTable. The thread which accesses the object locks it till it finishes its execution. Another thread can access the object only when the lock is released.

Synchronized Block

While creating synchronized methods within classes is an easy and effective means of achieving synchronization, it will not work in all cases. One such case is when; user wants to synchronize access to objects of a class that was not designed for multithreaded access. (i.e. the class whose object is intended to be synchronized does not have synchronized method.) Further, this class might have been created by a third party, and one would not have access to the source code to add synchronized to the appropriate method/s within the class.

In this situation, the solution would be to put calls to such method/s inside a synchronized block.

This is the general form of the synchronized block is:

```
synchronized(object) {  
// statements to be synchronized  
}
```

→ *synchronized(this)*

object is a reference to the object being synchronized.

Example:

```
public class Table {
```

```

        void printTable(int n){//method not synchronized   for(int
            i=1;i<=5;i++){
                System.out.println(n*i);
                try{
                    Thread.sleep(400);
                }catch(Exception e){System.out.println(e);}   }
            }
    }
}

```

```

class MyThread1 extends Thread{
    Table t;
    MyThread1(Table t){
        this.t=t;
    }
    public void run(){
        synchronized(t) {
            t.printTable(5);
        }
    }
}

```

```

class MyThread2 extends Thread{
    Table t;
    MyThread2(Table t){
        this.t=t;
    }
    public void run(){
        synchronized(t) {
            t.printTable(7);
        }
    }
}

```

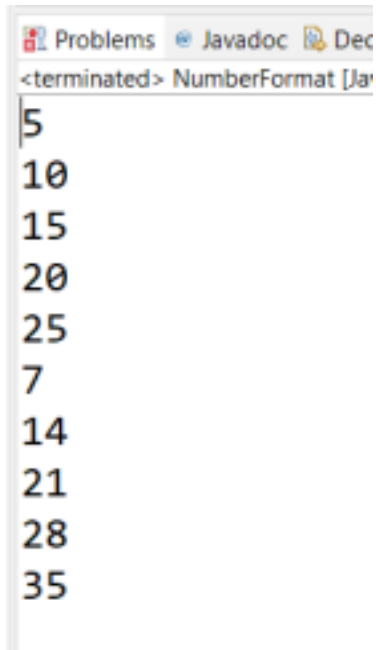
```

public class TestSynchronization1 { public static void
    main(String args[]){
        Table obj = new Table();//only one object
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
    }
}

```

```
        t1.start();
        t2.start();
    }
}
```

Output:



```
<terminated> NumberFormat [Ja
5
10
15
20
25
7
14
21
28
35
```

In the above example, the call to the printTable method is put in a synchronized block. And hence a synchronized output is obtained. That is the table of 5 is printed fully and is then followed by table of 7. The thread that takes control of an object locks it and releases it only when execution is completed.

The **io** package supports Java's basic I/O (input/output) system, including file I/O. Most programs cannot accomplish their goals without accessing external data. Data is retrieved from an *input* source. The results of a program are sent to an *output* destination. A network connection, memory buffer, or disk file can be manipulated by the Java I/O classes. Although physically different, these devices are all handled by the same abstraction: the *stream*. An I/O stream, is a logical entity that either produces or consumes information. An I/O stream is linked to a physical device by the Java I/O system. All I/O streams behave in the same manner, even if the actual physical devices they are linked to differ.

The Byte Streams

The byte stream classes provide a rich environment for handling byte-oriented I/O. A byte stream can be used with any type of object, including binary data. This versatility makes byte streams important to many types of programs. The Byte stream classes are child classes of the `InputStream` and `OutputStream` classes.

InputStream

`InputStream` is an abstract class that defines Java's model of streaming byte input. It implements the `AutoCloseable` and `Closeable` interfaces. Most of the methods in this class will throw an `IOException` when an I/O error occurs.

Some methods of `InputStream` are shown in the table below.

<code>int available()</code>	Returns the number of bytes of input currently available for reading.
<code>void close()</code>	Closes the input source. Further read attempts will generate an <code>IOException</code>
<code>int read()</code>	Returns an integer representation of the next available byte of input. <code>-1</code> is returned when the end of the file is encountered.
<code>int read(byte buffer[])</code>	Attempts to read up to <code>buffer.length</code> bytes into <code>buffer</code> and returns the actual number of bytes that were successfully read. <code>-1</code> is returned when the end of the file is encountered.
<code>int read(byte buffer[], int offset, int numBytes)</code>	Attempts to read up to <code>numBytes</code> bytes into <code>buffer</code> starting at <code>buffer[offset]</code> , returning the number of bytes successfully read. <code>-1</code> is returned when the end of the file is encountered.
<code>long skip(long numBytes)</code>	Ignores (that is, skips) <code>numBytes</code> bytes of input, returning the number of bytes actually ignored.

OutputStream

OutputStream is an abstract class that defines streaming byte output. It implements the AutoCloseable, Closeable, and Flushable interfaces. Most of the methods defined by this class return void and throw an IOException in the case of I/O errors.

Methods of OutputStream are shown in the table below.

void close()	void close() Closes the output stream. Further write attempts will generate an IOException.
void flush()	It flushes the output buffers.
void write(int b)	Writes a single byte to an output stream.
void write(byte buffer[])	Writes a complete array of bytes to an output stream.
void write(byte buffer[], int offset, int numBytes)	Writes a subrange of numBytes bytes from the array buffer, beginning at buffer[offset].

FileInputStream

The FileInputStream class creates an InputStream that can be used to read bytes from a file. Its two most common constructors are shown here:

```
FileInputStream(String filepath)
```

```
FileInputStream(File fileObj)
```

Either can throw a FileNotFoundException. Here, filepath is the full path name of a file, and fileObj is a File object that describes the file.

The following example creates two FileInputStreams that use the same disk file

```
FileInputStream f0 = new FileInputStream("/autoexec.bat");
```

```
File f = new File("/autoexec.bat");
```

```
FileInputStream f1 = new FileInputStream(f);
```

When a FileInputStream is created, it is also opened for reading.

```

1 package iopgms;
2 import java.io.*;
3 public class FileInputStreamEx {
4     public static void main(String args[]) throws IOException {
5
6         FileInputStream f1 = new FileInputStream("Example_1");
7         System.out.println("Total Available Bytes in Example_1: " + f1.available());
8         int n = f1.available();
9
10        System.out.println("Reading " + n + " bytes of the file using read()");
11        for (int i=0; i < n; i++) {
12            System.out.print((char) f1.read());
13        }
14        System.out.println();
15
16        FileInputStream f2 = new FileInputStream("Example_2");
17        System.out.println("Total Available Bytes in Example_2: " + f2.available());
18
19        n = f2.available();
20        System.out.println("Reading " + n + " bytes with read(b[])");
21        byte b[] = new byte[n];
22        f2.read(b);
23        for(byte x:b)
24            System.out.print((char)x);
25        System.out.println();
26
27        FileInputStream f3 = new FileInputStream("Example_3");
28        System.out.println("Total Available Bytes in Example_3: " + f3.available());
29        f3.skip(5);
30        System.out.println("Total Available Bytes after skip in Example_3: " + f3.available());
31    }
}

```

In the program shown above the methods read(), read(byte[] b), available(), skip() are used.

The output of the above code:

```

Total Available Bytes in Example_1: 16
Reading 16 bytes of the file using read()
Java programming
Total Available Bytes in Example_2: 15
Reading 15 bytes with read(b[])
C++ programming
Total Available Bytes in Example_3: 18
Total Available Bytes after skip in Example_3: 13

```

The text files used in the above program, Example_1, Example_2 and Example_3 contain the text “Java programming”, “C++ programming”, “Python Programming” respectively.

FileOutputStream

FileOutputStream creates an OutputStream that can be used to write bytes to a file. Its most commonly used constructors are:

FileOutputStream(String filePath)

FileOutputStream(File fileObj)

FileOutputStream(String filePath, boolean append)

FileOutputStream(File fileObj, boolean append)

These can throw a FileNotFoundException. Here, filePath is the full path name of a file, and fileObj is a File object that describes the file. If append is true, the file is opened in append mode. Creation of a FileOutputStream is not dependent on the file already existing. FileOutputStream will create the file before opening it for output when the object is created.

Sample Program:

```
import java.io.*;
public class FileOutputStreamEx {
    public static void main(String args[]) throws IOException {
        FileOutputStream f1 = new FileOutputStream("Example_1");
        String s = "This is Java Programming. It is object oriented";
        char ch[] = s.toCharArray();
        for (int i:ch) {
            f1.write((char)(i));
        }
        FileOutputStream f2 = new FileOutputStream("Example_2");
        s = "Hello World";
        byte b[] = s.getBytes();
        f2.write(b);
    }
}
```

The program uses the two variants of the write methods. write(int b) and write(byte[] b). The output in the files Example_1 and Example_2 are as below.

Example_1:

```
1|This is Java Programming. It is object oriented
```

Example_2:

```
1|Hello World
```

ByteArrayInputStream

The `ByteArrayInputStream` is composed of two words: `ByteArray` and `InputStream`. As the name suggests, it can be used to read byte array as input stream. Java `ByteArrayInputStream` class contains an internal buffer which is used to read byte array as stream. In this stream, the data is read from a byte array. The buffer of `ByteArrayInputStream` automatically grows according to data.

`ByteArrayInputStream` is an implementation of an input stream that uses a byte array as the source. This class has two constructors, each of which requires a byte array to provide the data source:

`ByteArrayInputStream(byte array[ ])`

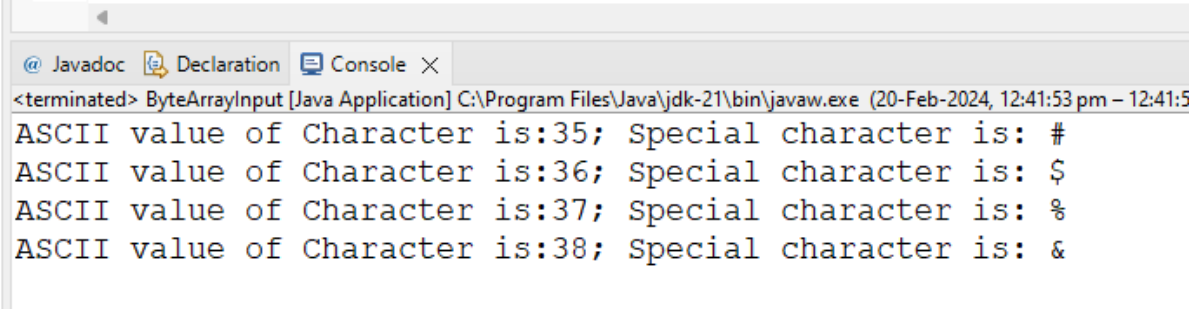
`ByteArrayInputStream(byte array[ ], int start, int numBytes)`

Here, `array` is the input source. The second constructor creates an `InputStream` from a subset of the byte array that begins with the character at the index specified by `start` and is `numBytes` long.

Ex 1:

```
import java.io.*;
public class ByteArrayInput {
    public static void main(String[] args) throws IOException {
        byte[] buf = { 35, 36, 37, 38 };
        // Create the new byte array input stream
        ByteArrayInputStream byt = new ByteArrayInputStream(buf);
        int k = 0;
        while ((k = byt.read()) != -1) {
            //Conversion of a byte into character
            char ch = (char) k;
            System.out.println("ASCII value of Character is:" + k + "; Special character is: " + ch);
        }
    }
}
```

O/p:

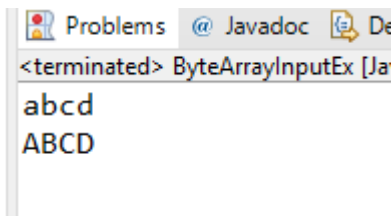


```
<terminated> ByteArrayInput [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (20-Feb-2024, 12:41:53 pm - 12:41:5
ASCII value of Character is:35; Special character is: #
ASCII value of Character is:36; Special character is: $
ASCII value of Character is:37; Special character is: %
ASCII value of Character is:38; Special character is: &
```

Ex 2:

```
package iopgms;
import java.io.*;
public class ByteArrayInputEx {
    public static void main(String[] args) throws IOException{
        String tmp = "abcd";
        byte b[] = tmp.getBytes();
        ByteArrayInputStream ba = new ByteArrayInputStream(b);
        for(int i = 0; i<2; i++) {
            int c;
            while((c= ba.read())!=-1) {
                if(i==0)
                    System.out.print((char)c);
                else
                    System.out.print(Character.toUpperCase((char)c));
            }
            System.out.println();
            ba.reset();
        }
    }
}
```

O/p:



```
<terminated> ByteArrayInputEx [Ja
abcd
ABCD
```

In the example each character is read and printed in lowercase. The stream is then reset and read again, this time converting each character to upper case before printing.

ByteArrayOutputStream

ByteArrayOutputStream is an implementation of an output stream that uses a byte array as the destination. ByteArrayOutputStream has two constructors, shown here:

ByteArrayOutputStream()

ByteArrayOutputStream(int numBytes)

In the first form, a buffer of 32 bytes is created. In the second, a buffer is created with a size equal to that specified by numBytes. The buffer is held in the protected buf field of

ByteArrayOutputStream. The buffer size will be increased automatically, if needed. The number of bytes held by the buffer is contained in the protected count field of ByteArrayOutputStream.

```
package iopgms;
import java.io.*;
public class ByteArrayOutputEx {

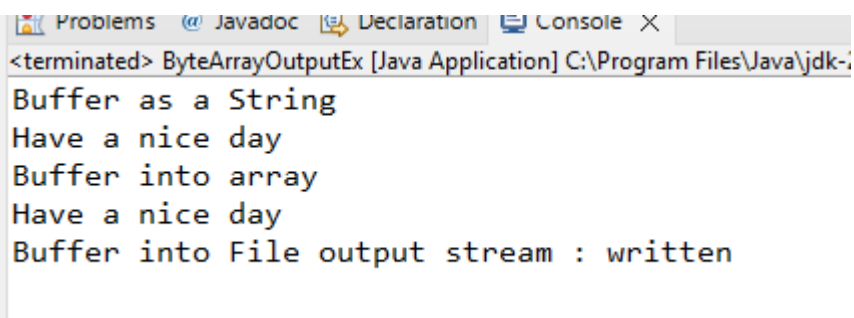
    public static void main(String[] args) throws IOException {
        ByteArrayOutputStream ba = new ByteArrayOutputStream();
        String s = "Have a nice day";
        byte buf[] = s.getBytes();
        ba.write(buf);

        System.out.println("Buffer as a String");
        System.out.println(ba.toString());

        System.out.println("Buffer into array");
        byte c[] = ba.toByteArray();
        for(byte i:c)

            System.out.print((char)(i));
        System.out.println();
        System.out.println("Buffer into File output stream : written");
        FileOutputStream fos = new FileOutputStream("Example_1");
        ba.writeTo(fos);
    }
}
```

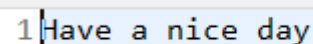
O/p:



The screenshot shows a Java IDE console window with the following output:

```
<terminated> ByteArrayOutputEx [Java Application] C:\Program Files\Java\jdk-:
Buffer as a String
Have a nice day
Buffer into array
Have a nice day
Buffer into File output stream : written
```

Example_1:



The screenshot shows a text file named "Example_1" containing the following text:

```
1 Have a nice day
```

In the above example data is written once into a buffer. Then buffer is printed as a string, transferred to an array and finally to a file output stream. Appropriate print statements are included in the program for better understanding.

Filtered Byte Streams

Filtered streams are simply wrappers around underlying input or output streams that transparently provide some extended level of functionality. The filtered byte streams are `FilterInputStream` and `FilterOutputStream`. Their constructors are shown here:

```
FilterOutputStream(OutputStream os)
```

```
FilterInputStream(InputStream is)
```

The methods provided in these classes are identical to those in `InputStream` and `OutputStream`.

Buffered Byte Streams

For the byte-oriented streams, a buffered stream extends a filtered stream class by attaching a memory buffer to the I/O streams. This buffer allows Java to do I/O operations on more than a byte at a time, hence increasing performance. The buffered byte stream classes are `BufferedInputStream` and `BufferedOutputStream`. `PushbackInputStream`.

BufferedInputStream

Java's `BufferedInputStream` class allows to "wrap" any `InputStream` into a buffered stream and achieves performance improvement. `BufferedInputStream` has two constructors:

```
BufferedInputStream(InputStream inputStream)
```

```
BufferedInputStream(InputStream inputStream, int bufSize)
```

The first form creates a buffered stream using a default buffer size. In the second, the size of the buffer is passed in `bufSize`.

The example program shows the use of `mark` and `reset` method. 3 bytes are read from the array and `mark()` is called. When `reset()` is called, the stream read starts from the mark point and not the beginning. `mark` expects an `int` parameter, `readLimit`. The `readLimit` argument tells this input stream to allow that many bytes to be read before the mark position gets

invalidated. (This parameter does not have any effect even if changed in the sample program below)

```
package iopgms;
import java.io.*;
public class BufferedInputStream {

    public static void main(String[] args) throws IOException {
        String s = "Have a nice day";
        byte buf[] = s.getBytes();
        ByteArrayInputStream ba = new ByteArrayInputStream(buf);
        BufferedInputStream bi = new BufferedInputStream(ba);

        System.out.println("Char : "+(char)bi.read());
        System.out.println("Char : "+(char)bi.read());
        System.out.println("Char : "+(char)bi.read());

        bi.mark(3);
        System.out.println("Char : "+(char)bi.read());
        System.out.println("Char : "+(char)bi.read());
        System.out.println("Char : "+(char)bi.read());
        bi.reset();
        System.out.println("Char : "+(char)bi.read());
        System.out.println("Char : "+(char)bi.read());
    }
}
```

O/p:

```
terminated: bufferedinp
Char : H
Char : a
Char : v
Char : e
Char : 
Char : a
Char : e
Char :
```

Characters H, a , v is read after which mark is called. 3 characters are read again ie e, “ ”, and a. On reset reading happens from the marked point that is “e” onwards and not from “H”.

BufferedOutputStream

Java `BufferedOutputStream` class is used for buffering an output stream. It internally uses buffer to store data. It adds more efficiency than to write data directly into a stream. So, it makes the performance fast. A `BufferedOutputStream` is similar to any `OutputStream` with the exception of an added `flush( )` method that is used to ensure that data buffers are physically written to the actual output device.

Here are the two available constructors:

`BufferedOutputStream(OutputStream outputStream)`

`BufferedOutputStream(OutputStream outputStream, int bufSize)`

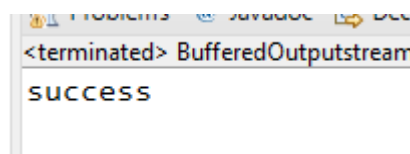
The first form creates a buffered stream using the default buffer size. In the second form, the size of the buffer is passed in `bufSize`.

Ex:

```
import java.io.*;
public class BufferedOutputstreamEx {

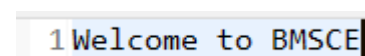
    public static void main(String[] args) throws IOException{
        FileOutputStream fout=new FileOutputStream("Example_3");
        BufferedOutputStream bout=new BufferedOutputStream(fout);
        String s="Welcome to BMSCE";
        byte b[]=s.getBytes();
        bout.write(b);
        bout.flush();
        System.out.println("success");
    }
}
```

O/p:



The screenshot shows a Java IDE window titled "Example_3" with a console output area. The output area displays the text "<terminated> BufferedOutputstream" followed by "success" on the next line.

Example_3



The screenshot shows a text editor window with the text "1 Welcome to BMSCE" on the first line.

flush() is used for performance reasons. Flushing an output stream means that the user would want to stop, wait for the content of the stream to be completely transferred to its destination, and then resume execution with the stream empty and the content sent.

PushbackInputStream

Pushback is used on an input stream to allow a byte to be read and then returned (that is, “pushed back”) to the stream. The PushbackInputStream class implements this idea. It provides a mechanism to “peek” at what is coming from an input stream without disrupting it. PushbackInputStream has the following constructors:

PushbackInputStream(InputStream inputStream)

PushbackInputStream(InputStream inputStream, int numBytes)

The first form creates a stream object that allows one byte to be returned to the input stream. The second form creates a stream that has a pushback buffer that is numBytes long. This allows multiple bytes to be returned to the input stream. PushbackInputStream provides unread() method. This is used to push back the already read character back into the stream.

```
import java.io.*;
public class PushBackInputStreamEx {

    public static void main(String[] args) throws IOException {
        String srg = "1##2#34###12";
        byte ary[] = srg.getBytes();
        ByteArrayInputStream array = new ByteArrayInputStream(ary);
        PushbackInputStream push = new PushbackInputStream(array);
        int i;
        while( (i = push.read())!= -1) {
            if(i == '#') {
                int j;
                if( (j = push.read()) == '#'){
                    System.out.print("***");
                }
            }
        }
    }
}
```

```

        }else {
            push.unread(j);
            System.out.print((char)i);
        }
    }else {
        System.out.print((char)i);
    }
}
}
}
}

```

O/p:

```

<terminated> PushBackInputStri
1**2#34**#12

```

In the program, “##” is replaced by “**”. The replacement occurs only when there are two hashes consecutively. Notice that when 5th character “#” is encountered, the 6th character “3” is read. Since it is not “#” it is pushed back to the stream using the unread method. The same is the case when the 10th character “#” and the 11th character “1” is read.

SequenceInputStream

The SequenceInputStream class allows to concatenate multiple InputStreams. The construction of a SequenceInputStream is different from any other InputStream. A SequenceInputStream constructor uses of InputStreams as its argument.

SequenceInputStream(InputStream first, InputStream second)

The class fulfills read requests from the first InputStream until it runs out and then switches over to the second one.

Example:

```
import java.io.*;
public class SequenceInputStreamEx {

    public static void main(String[] args) throws IOException {
        FileInputStream input1=new FileInputStream("Example_1");
        FileInputStream input2=new FileInputStream("Example_2");
        SequenceInputStream inst=new SequenceInputStream(input1, input2);
        int j;
        while((j=inst.read())!=-1){
            System.out.print((char)j);
        }
        inst.close();
        input1.close();
        input2.close();
    }
}
```

The two files Example_1 and Example_2 contain the data “Have a nice day.” and “Hello World.” respectively. In the program the SequenceInputStream object uses the read method to read from both files in sequence.

O/p:

```
Have a nice day.Hello World.
```

Print Stream

The PrintStream class provides methods to write data to another stream. The PrintStream class automatically flushes the data. There is no need to call flush() method. Moreover, its methods don't throw IOException. PrintStream has various overloaded versions of printf that can be used to print any kind of data. (Refer to oracle docs for the list of methods supported.)

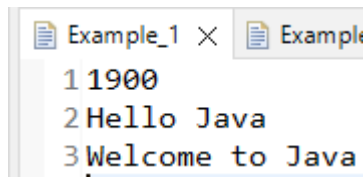
Ex 1:

```
import java.io.*;
public class PrintStreamEx1 {

    public static void main(String[] args) throws IOException {
        FileOutputStream fout=new FileOutputStream("Example_1");
        PrintStream pout=new PrintStream(fout);
        pout.println(1900);
        pout.println("Hello Java");
        pout.println("Welcome to Java");
    }
}
```

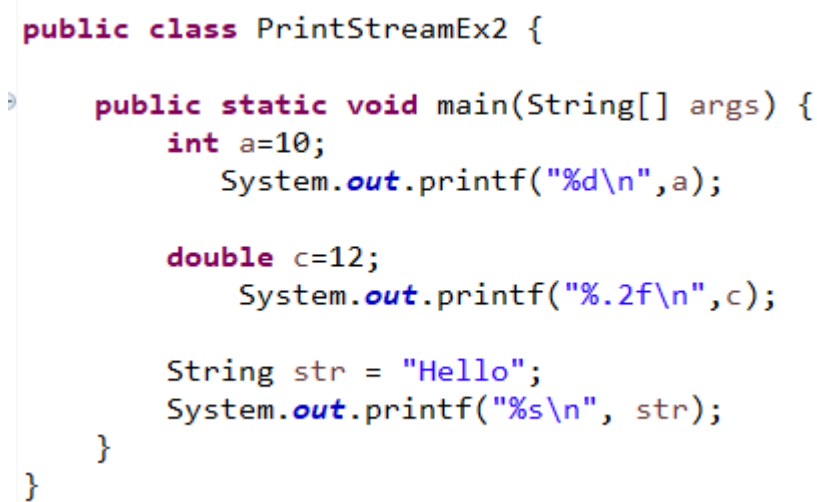
Output:

(Contents printed to file Example_1)



```
Example_1 x Example_1
1 1900
2 Hello Java
3 Welcome to Java
```

With the release of JDK 5, the `printf()` method was added to `PrintStream`. It allows the user to specify the precise format of the data to be written. (Same as in C/C++)

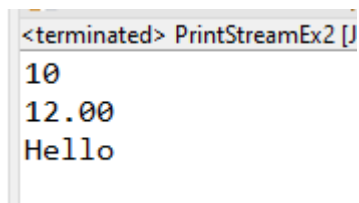


```
public class PrintStreamEx2 {
    public static void main(String[] args) {
        int a=10;
        System.out.printf("%d\n",a);

        double c=12;
        System.out.printf("%.2f\n",c);

        String str = "Hello";
        System.out.printf("%s\n", str);
    }
}
```

O/p:



```
<terminated> PrintStreamEx2 [J
10
12.00
Hello
```

DataOutputStream and DataInputStream

A data input stream lets an application read primitive Java data types from an underlying input stream in a machine-independent way. An application uses a data output stream to write data that can later be read by a data input stream. `DataInputStream` is not necessarily safe for multithreaded access.

Java `DataOutputStream` class allows an application to write primitive Java data types to the output stream in a machine-independent way. Java application generally uses the data output stream to write data that can later be read by a data input stream.

`DataOutputStream` and `DataInputStream` implement the `DataOutput` and `DataInput` interfaces, respectively.

Example:

```
import java.io.*;

public class DataInputStreamEx {

    public static void main(String[] args) throws IOException{
        FileOutputStream fout = new FileOutputStream("Example_1");
        DataOutputStream out = new DataOutputStream(fout);
        out.writeDouble(98.6);
        out.writeInt(1000);
        out.writeBoolean(true);
        out.close();
        FileInputStream fin = new FileInputStream("Example_1");
        DataInputStream in = new DataInputStream(fin);
        double d = in.readDouble();
        int i = in.readInt();
        boolean b = in.readBoolean();
        System.out.println("Here are the values: " + d + " " + i + " " + b);
        in.close();
    }
}
```

The program uses `DataOutputStream` object to write into a file. Note the use of methods `writeDouble()`, `writeInt()` and `writeBoolean()` instead of just `write()`. It uses the `DataInputStream` object to read from file. Note the use of methods `readDouble()`, `readInt()` and `readBoolean()` instead of just `read()`.

RandomAccessFile

`RandomAccessFile` encapsulates a random-access file. It is not derived from `InputStream` or `OutputStream`. Instead, it implements the interfaces `DataInput` and `DataOutput`. `RandomAccessFile` is special because it supports positioning requests—that is, the file pointer can be positioned within the file. Commonly used constructor is:

`RandomAccessFile(String filename, String access)` throws `FileNotFoundException`

The method `seek( )`, is used to set the current position of the file pointer within the file:

`void seek(long newPos)` throws `IOException`

Here, newPos specifies the new position, in bytes, of the file pointer from the beginning of the file. After a call to seek(), the next read or write operation will occur at the new file position.

Example:

```
package iopgms;
import java.io.*;
public class RandomAccessFileEx {

    public static void main(String[] args) throws IOException {
        RandomAccessFile raf = new RandomAccessFile("Example_3", "rw");
        raf.seek(1);
        byte[] bytes = new byte[5];
        raf.read(bytes);

        System.out.println(new String(bytes));

        raf.seek(5);
        raf.write("Data".getBytes());

        raf.seek(raf.length());
        raf.write("Bull temple road".getBytes());
    }
}
```

The file Example_3 contains “Welcome to BMSCE” initially. The program opens the file in read and write mode. seek(1) helps in moving the file pointer by one. So the pointer is now at “e”. read(bytes) reads 5 characters from “e”. The characters “elcom” is read. Next raf.seek(5) moves the pointer by 5(from the start). The file pointer is now at character “o”. From here “Data” is written. Append option isn’t available in RandomAccessFile. The pointer can be seeked till the end and then write can be called to achieve append operation.

Final Output as seen in the file Example_3:

```
1 WelcoDatao BMSCEBull temple road
```